DAY 5

Akash J | 192324295

Dynamic Programming – Optimization Problems

---

1. 0/1 Knapsack Problem (Dynamic Programming)

Aim:

To find the maximum profit that can be obtained by selecting items with given weights and values without exceeding the knapsack capacity using dynamic programming.

Algorithm:

1. Create a 2D DP table.
2. Iterate through items and capacities.
3. If the item weight is less than or equal to current capacity, take the max of including or excluding it.
4. Otherwise, exclude it. Return the bottom-right cell.

Code:

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
dp[i - 1][w])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]
```

Input:

```
weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
```

Output:

```
Maximum Profit: 9
```

Result: Maximum profit computed successfully using bottom-up DP.

2. Travelling Salesperson Problem (Dynamic Programming)

Aim:

To find the shortest possible route that visits every city exactly once and returns to the origin city using DP and memoization.

Algorithm:

1. Represent visited cities using a bitmask.
2. Recursively try visiting all unvisited cities.
3. Cache results to avoid redundant calculations.
4. Return the minimum cost.

Code:

```
from functools import lru_cache

def tsp(graph):
    n = len(graph)
    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]
        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(ans, graph[pos][city] + visit(mask | (1 << city),
city))
        return ans
    return visit(1, 0)
```

Input:

```
graph = [
[0, 10, 15, 20],
[10, 0, 35, 25],
[15, 35, 0, 30],
[20, 25, 30, 0]
]
```

Output:

```
Minimum Cost: 80
```

Result: Minimum TSP cost found successfully using recursion and memoization.

3. Optimal Binary Search Tree (OBST)

Aim:

To construct a binary search tree that minimizes the expected search cost given keys and their search frequencies.

Algorithm:

1. Create a 2D cost table.
2. Initialize diagonal elements with individual frequencies.
3. For increasing chain lengths, compute the minimum cost by trying every key as the root.

Code:

```
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total_freq = sum(freq[i:j + 1])
            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(cost[i][j], left + right + total_freq)
    return cost[0][n - 1]
```

Input:

```
keys = [10, 20, 30]
freq = [34, 8, 50]
```

Output:

```
Optimal BST Cost: 142
```

Result: Optimal BST cost correctly calculated.

4. 0/1 Knapsack Problem Program

Aim:

To solve the 0/1 Knapsack problem iteratively using a bottom-up DP table.

Algorithm:

1. Initialize a 2D DP array with zeros.
2. Loop over items and capacities.

3. For each cell, take the maximum value of picking or leaving the current item.
4. Return the max value for the full capacity.

Code:

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
```

Input:

```
wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
```

Output:

```
Maximum Profit = 9
```

Result: Maximum profit correctly evaluated using iterative DP.

5. 0/1 Knapsack Using Memoization

Aim:

To solve the 0/1 Knapsack problem efficiently using a top-down approach with memoization.

Algorithm:

1. Create a memoization table initialized with -1.
2. Base case: return 0 if no items or 0 capacity.
3. If already computed, return the cached value.
4. Recursively compute and store the max of including or excluding the item.

Code:

```
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0
    if memo[n][W] != -1:
        return memo[n][W]
    if wt[n-1] <= W:
        memo[n][W] = max(
            val[n-1] + knapsack(W-wt[n-1], wt, val, n-1, memo),
            knapsack(W, wt, val, n-1, memo)
        )
    else:
        memo[n][W] = knapsack(W, wt, val, n-1, memo)
    return memo[n][W]
```

Input:

```
wt = [1,3,4,5]
val = [1,4,5,7]
W = 7
```

Output:

```
Maximum Profit = 9
```

Result: Memoized top-down knapsack solution executed optimally.

6. Subset Sum Problem

Aim:

To determine if there is a subset of the given array whose sum equals the target value.

Algorithm:

1. Create a boolean DP table of size (n+1) x (target+1).
2. Set the first column to True (sum 0 is always possible).
3. Fill the table by checking if the current element can be included or excluded to reach the target sum.

Code:

```
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in range(n+1)]
    for i in range(n+1):
        dp[i][0] = True
    for i in range(1, n+1):
        for j in range(1, target+1):
            if arr[i-1] <= j:
                dp[i][j] = dp[i-1][j] or dp[i-1][j-arr[i-1]]
            else:
                dp[i][j] = dp[i-1][j]
    return dp[n][target]
```

Input:

```
arr = [3, 34, 4, 12, 5, 2]
target = 9
```

Output:

```
True
```

Result: Subset sum presence successfully verified.

7. Travelling Salesperson Problem (DP Version)

Aim:

To find the minimum travel cost for visiting all nodes exactly once using DP with bitmasking.

Algorithm:

1. Define a recursive function with the current city and a bitmask of visited cities.
2. If all cities are visited, return the cost to the start.
3. Otherwise, recursively calculate the cost for unvisited cities.

Code:

```
from functools import lru_cache

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
n = len(graph)

@lru_cache(None)
def tsp(mask, pos):
    if mask == (1<
```

Input:

```
tsp(1, 0)
```

Output:

```
Minimum Cost = 80
```

Result: TSP using bitmasking evaluated optimally.

8. Travelling Salesperson Using DP Table

Aim:

To solve the Travelling Salesperson Problem using an iterative bottom-up DP approach with bitmasking.

Algorithm:

1. Initialize a DP table of size `2^n x n` with infinity.
2. Set the starting state to 0.
3. Iterate over all masks and compute the shortest path for adding a new unvisited city.
4. Add the return cost to the origin and find the minimum.

Code:

```
def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')]*n for _ in range(1<
```

Input:

```
graph = [
[0,10,15,20],
[10,0,35,25],
[15,35,0,30],
[20,25,30,0]
]
```

Output:

```
Minimum Cost = 80
```

Result: Iterative DP solution for TSP successfully computed.

9. Optimal Binary Search Tree (OBST) Program

Aim:

To find the optimal cost of a Binary Search Tree using dynamic programming given only the frequencies.

Algorithm:

1. Initialize a cost matrix.
2. Iterate through substring lengths from 2 to n.
3. For each range, compute the cost of making each node the root and choose the minimum.

Code:

```
def optimal_bst(freq):
    n = len(freq)
    cost = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for L in range(2, n+1):
        for i in range(n-L+1):
            j = i+L-1
            cost[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = cost[i][r-1] if r > i else 0
                right = cost[r+1][j] if r < j else 0
                cost[i][j] = min(cost[i][j], left + right + total)
    return cost[0][n-1]
```

Input:

```
freq = [34, 8, 50]
```

Output:

```
Optimal Cost = 142
```

Result: Optimal tree cost identified successfully.

10. Matrix Chain Multiplication

Aim:

To find the most efficient way to multiply a given sequence of matrices to minimize scalar multiplications.

Algorithm:

1. Create a 2D DP table.
2. Consider all chain lengths from 2 to n.
3. For each segment, try every possible split point k.
4. Record the minimum multiplication cost in the DP table.

Code:

```
def matrix_chain(arr):
    n = len(arr)
    dp = [[0]*n for _ in range(n)]
    for L in range(2, n):
        for i in range(1, n-L+1):
            j = i+L-1
            dp[i][j] = float('inf')
            for k in range(i, j):
                q = dp[i][k] + dp[k+1][j] + arr[i-1]*arr[k]*arr[j]
                dp[i][j] = min(dp[i][j], q)
    return dp[1][n-1]
```

Input:

```
arr = [10, 20, 30, 40, 30]
```

Output:

```
Minimum Multiplications = 30000
```

Result: Matrix chain multiplications minimized successfully.

11. Coin Change (Minimum Coins)

Aim:

To find the minimum number of coins required to make a given amount.

Algorithm:

1. Initialize a DP array of size `amount + 1` with infinity.
2. Set dp[0] = 0.
3. For each amount up to the target, try every coin and update the minimum coins needed.

Code:

```
def min_coins(coins, amount):
    dp = [float('inf')] * (amount+1)
    dp[0] = 0
    for i in range(1, amount+1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i-c]+1)
    return dp[amount]
```

Input:

```
coins = [1, 2, 5]
amount = 11
```

Output:

```
Minimum Coins = 3
```

Result: Minimum coins derived correctly using DP array.

12. Rod Cutting Problem

Aim:

To determine the maximum revenue obtainable by cutting up a rod and selling the pieces.

Algorithm:

1. Initialize a DP array to store max revenue for each length.
2. Iterate over all lengths from 1 to n.
3. For each length, try all possible cuts and find the maximum revenue.

Code:

```
def rod_cut(price, n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        for j in range(i):
            dp[i] = max(dp[i], price[j] + dp[i-j-1])
    return dp[n]
```

Input:

```
price = [1, 5, 8, 9, 10, 17, 17, 20]
n = 8
```

Output:

```
Maximum Revenue = 22
```

Result: Maximum revenue computed optimally.

13. Minimum Cost Path

Aim:

To find the minimum cost to reach the bottom-right cell of a grid from the top-left cell.

Algorithm:

1. Create a 2D DP array.
2. Initialize the first cell, then populate the first row and first column.
3. Fill the rest of the cells by adding the current cost to the minimum of the top or left cell.

Code:

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = cost[0][0]
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] + cost[i][0]
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] + cost[0][j]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = cost[i][j] + min(dp[i-1][j], dp[i][j-1])
    return dp[m-1][n-1]
```

Input:

```
cost = [
[1, 3, 1],
[1, 5, 1],
[4, 2, 1]
]
```

Output:

```
Minimum Cost = 7
```

Result: Minimum path cost found accurately.